

Universidad Tecnológica de Honduras (UTH)

Asignatura:

Programacion Movil I

Tema:

Tarea 2.2 API REST Bitácora

Alumno y Número de Cuenta:

Angel Manuel Rivera 202330010471

Anthony Joel Gonzales Juárez 202310010114

Brayan Wilfredo Torres Aguilar 201820030041

Johan Moises Lopez Moran 202220010502

Luis David Diaz Medina 202210010189

Master:

Ricardo Enrique Lagos Mendoza

Fecha:

07 de noviembre del 2026

Introducción

Hoy en día es fundamental que las aplicaciones compartan información de forma ordenada. En esta tarea desarrollamos una API con C# y ASP.NET Core, organizada por capas con arquitectura limpia, para guardar y administrar los registros de nuestra bitácora de campo. Esta API será el puente entre la base de datos y la aplicación Android que usaremos después. Todo se probó, documentó y se guardó en GitHub para llevar un control del trabajo en equipo.

Objetivos

Objetivo General

Crear una API REST bien estructurada que permita gestionar los registros de la bitácora, con validaciones, manejo de errores, documentación y pruebas, lista para conectarse con la aplicación móvil.

Objetivos Específicos

- Separar el proyecto en las capas: dominio, aplicación, infraestructura y presentación.
- Definir los datos de cada registro y programar las operaciones de crear, ver, modificar y eliminar.
- Revisar que la información sea correcta y manejar los errores sin exponer datos del servidor.
- Documentar con Swagger y probar todo en Postman.
- Subir el código a GitHub con instrucciones claras de uso.

1. **Diagrama de capas y dependencias**

Bitacora.Api --> Bitacora.Application

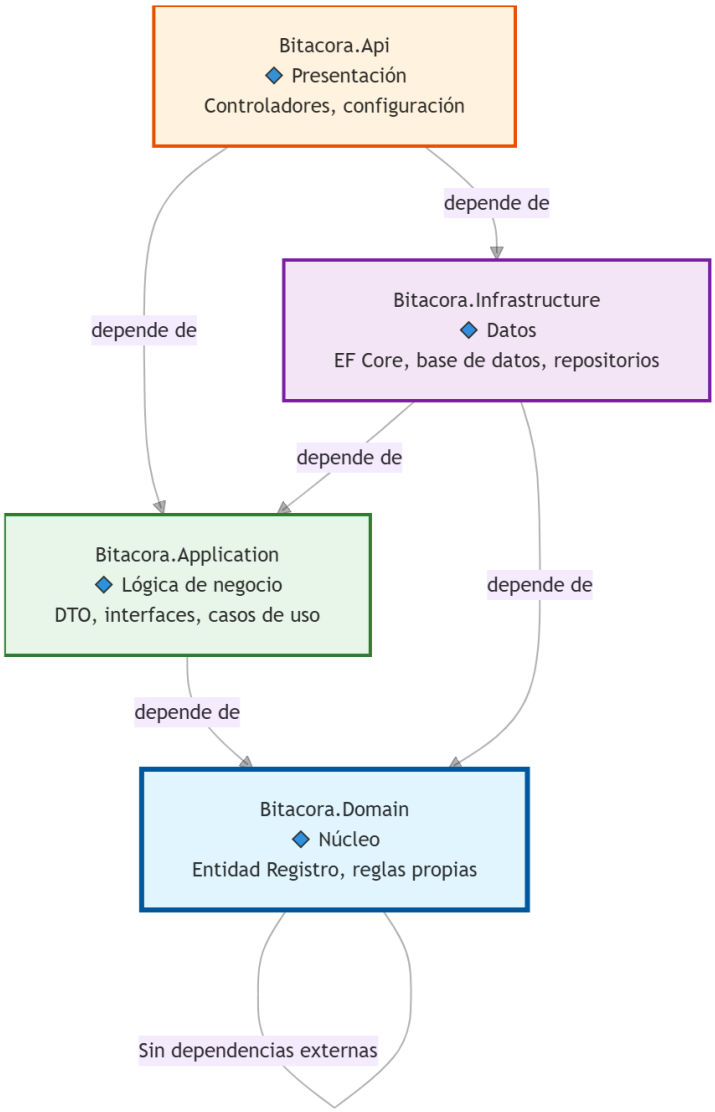
Bitacora.Api --> Bitacora.Infrastructure

Bitacora.Infrastructure --> Bitacora.Application

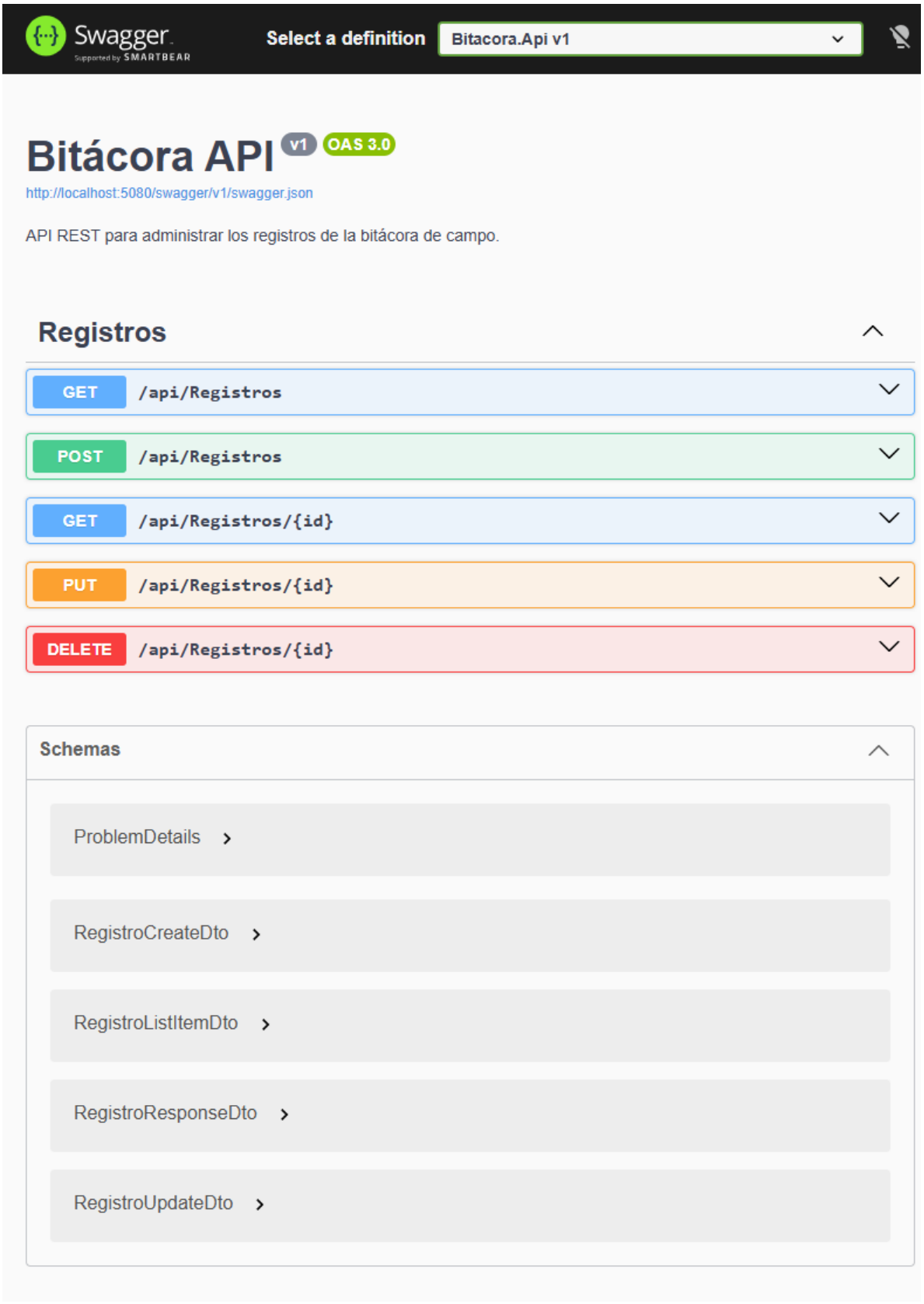
Bitacora.Infrastructure --> Bitacora.Domain

Bitacora.Application --> Bitacora.Domain

Bitacora.Domain --> (ninguna dependencia)



2. **Captura de Swagger**



3. **Pruebas en Postman (exitosas y con error)**

   Colección: docs/Bitacora.postman_collection.json

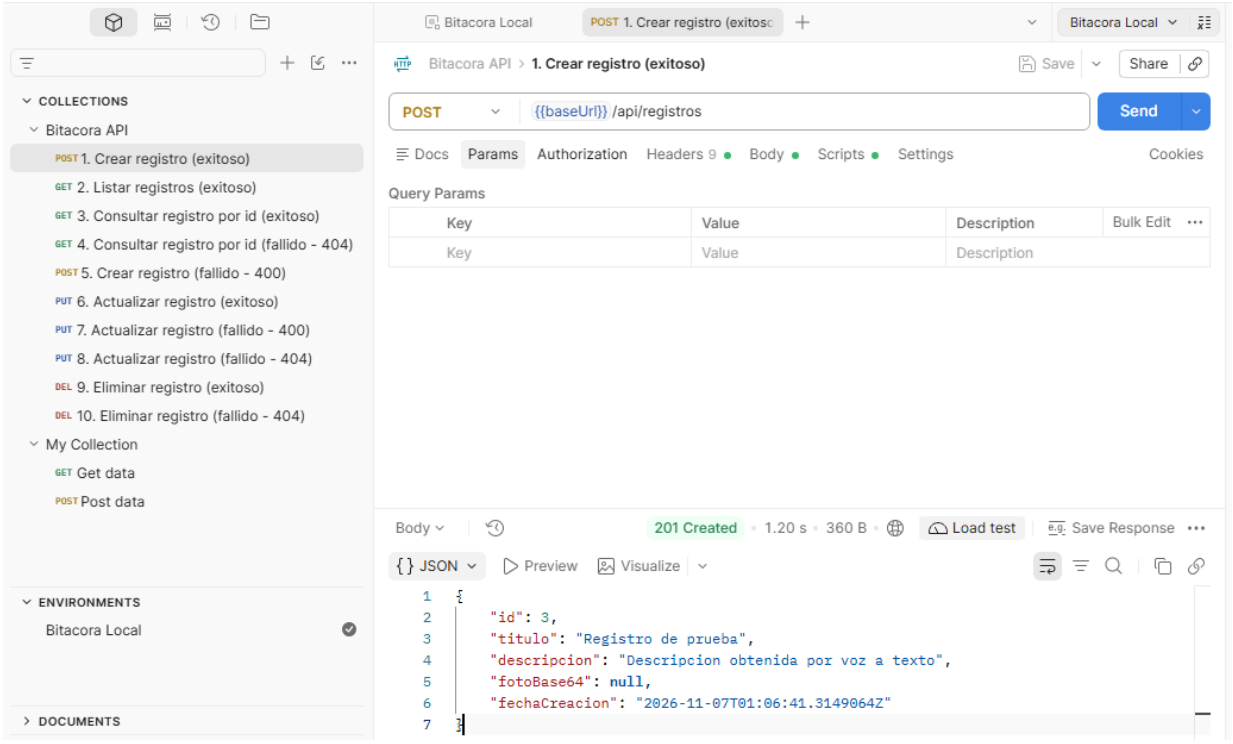
   Entorno:

   docs/Bitacora.postman_environment.json(variablebaseUrl = http://localhost:5080).

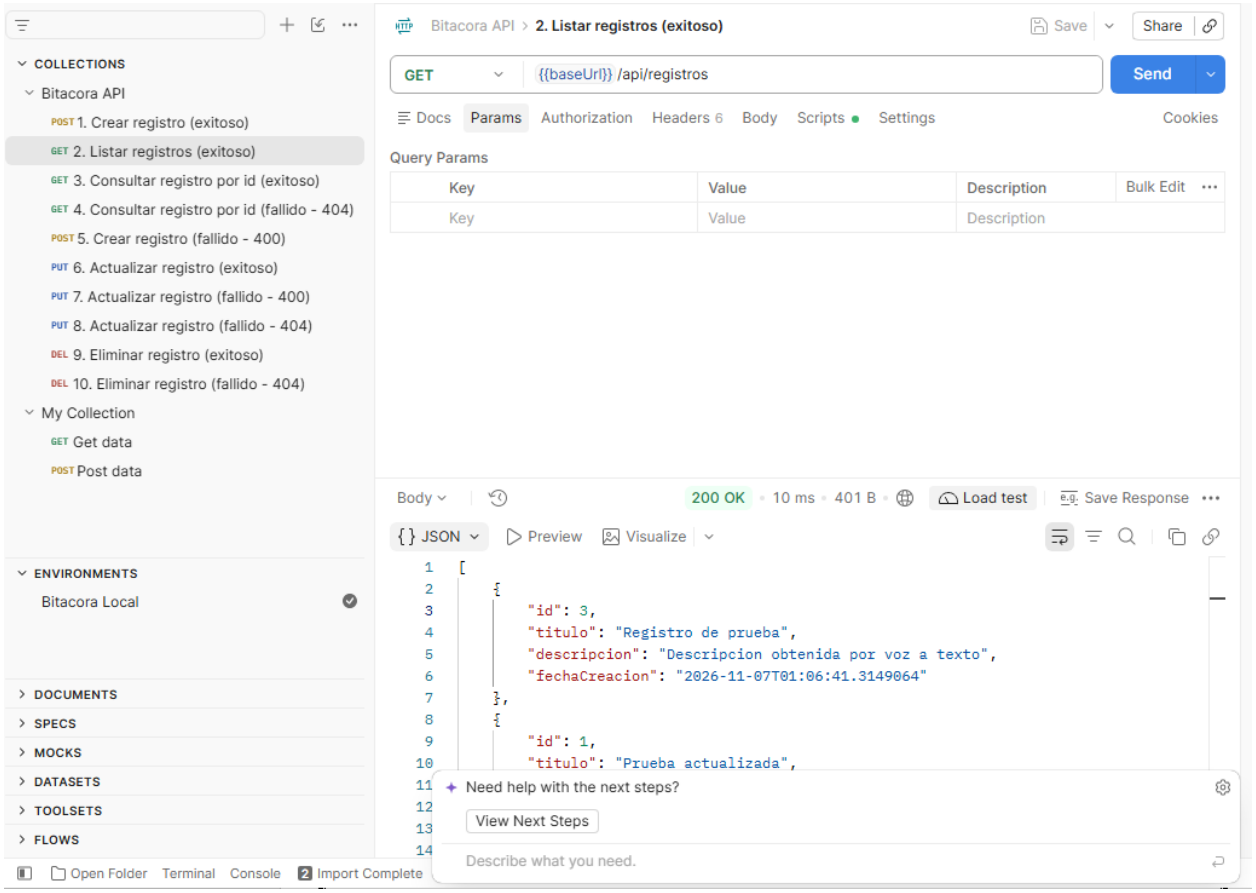
   - GET /api/registros exitoso (200), sin foto.
   - GET /api/registros/{id} exitoso (200) y fallido (404).
   - POST /api/registros exitoso (201 con Location) y fallido (400).
   - PUT /api/registros/{id} exitoso (200), fallido (400) y fallido (404).
   - DELETE /api/registros/{id} exitoso (204) y fallido (404).

4. **Capturas de pruebas en Postman**

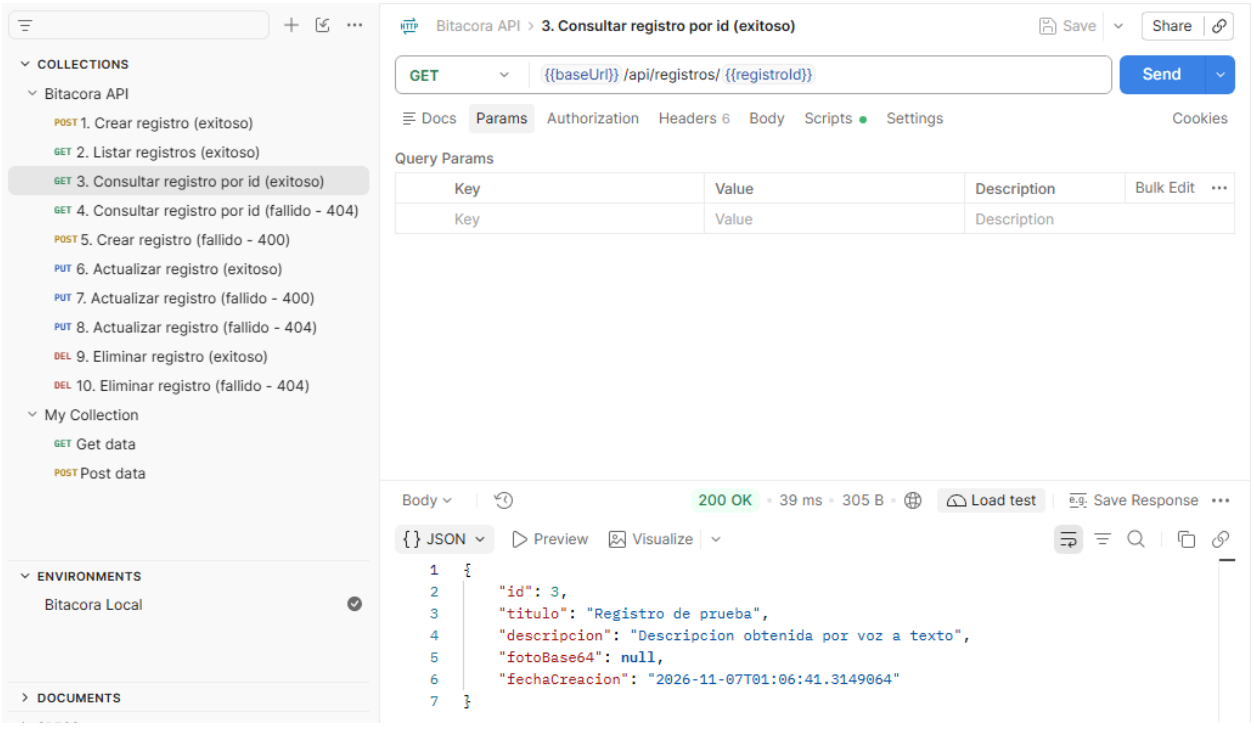
Crear registro (exitoso)



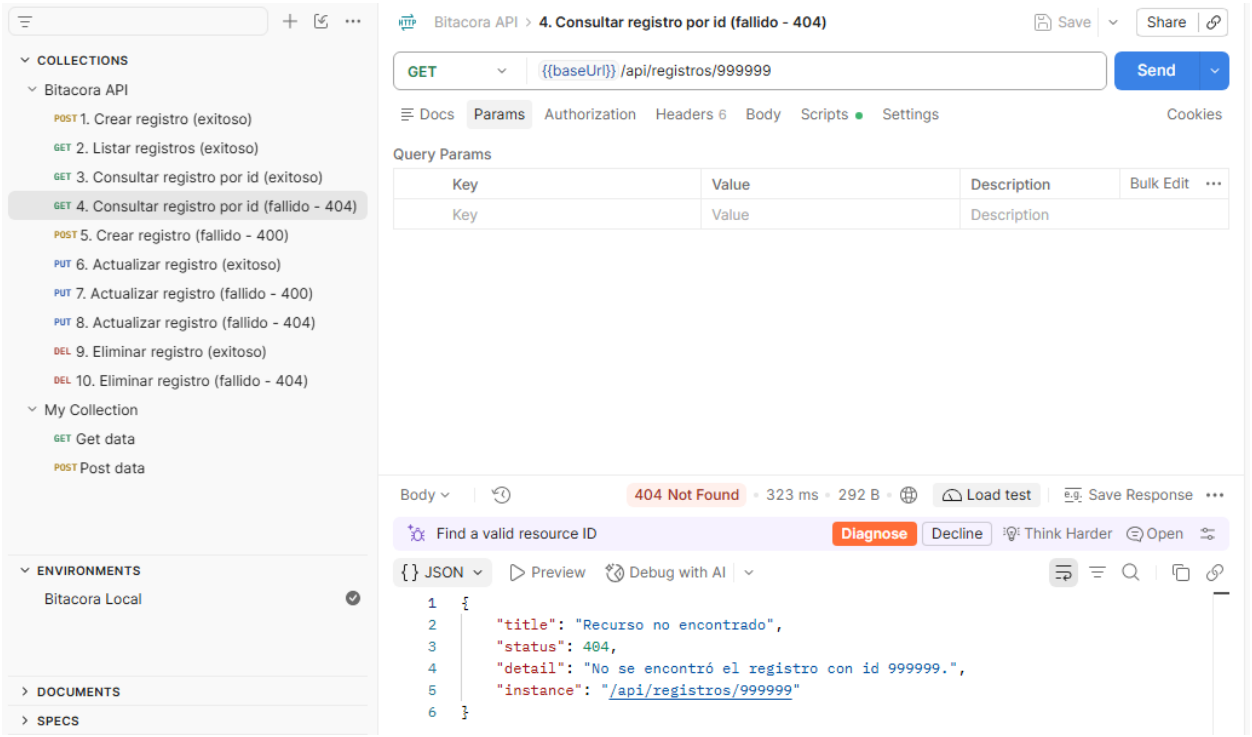
Listar registros (exitoso)



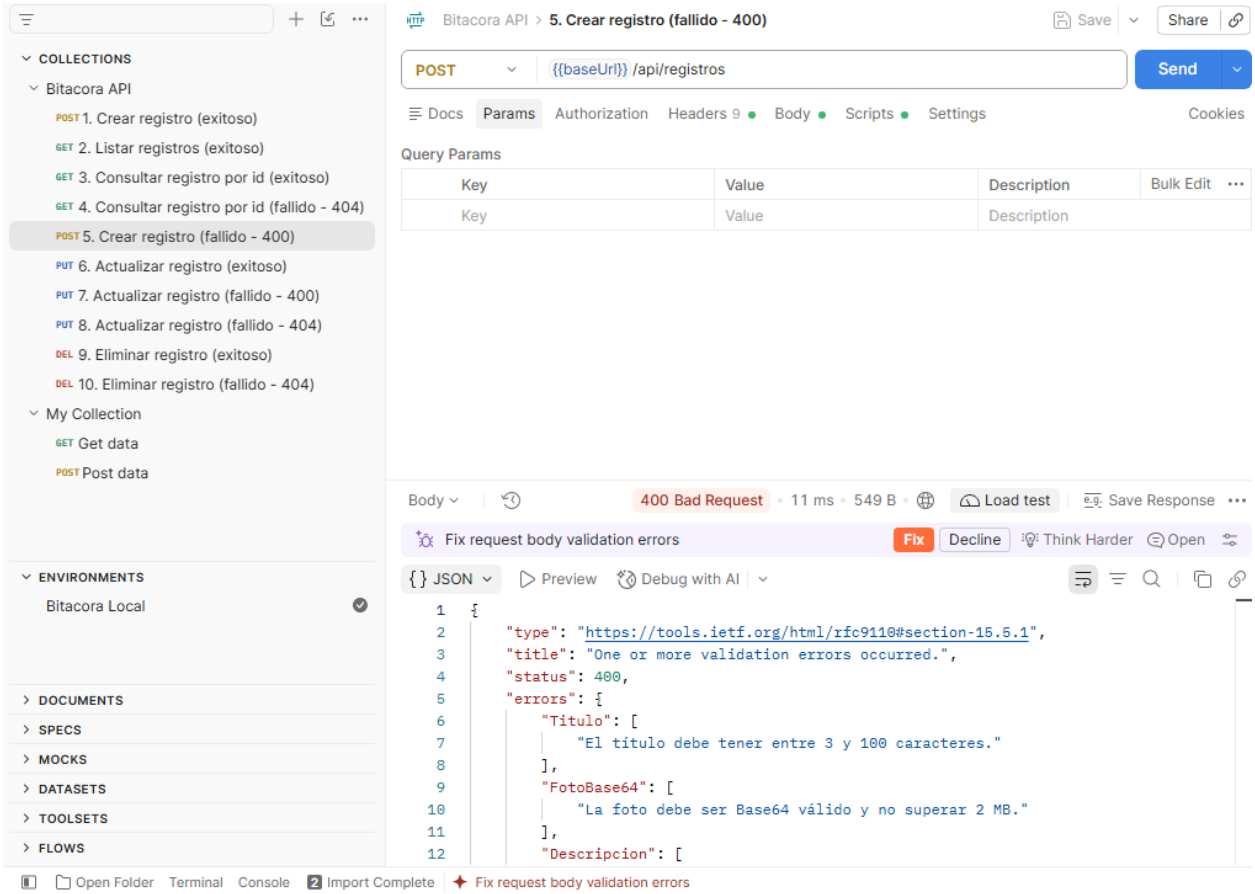
Consultar registro por id (exitoso)



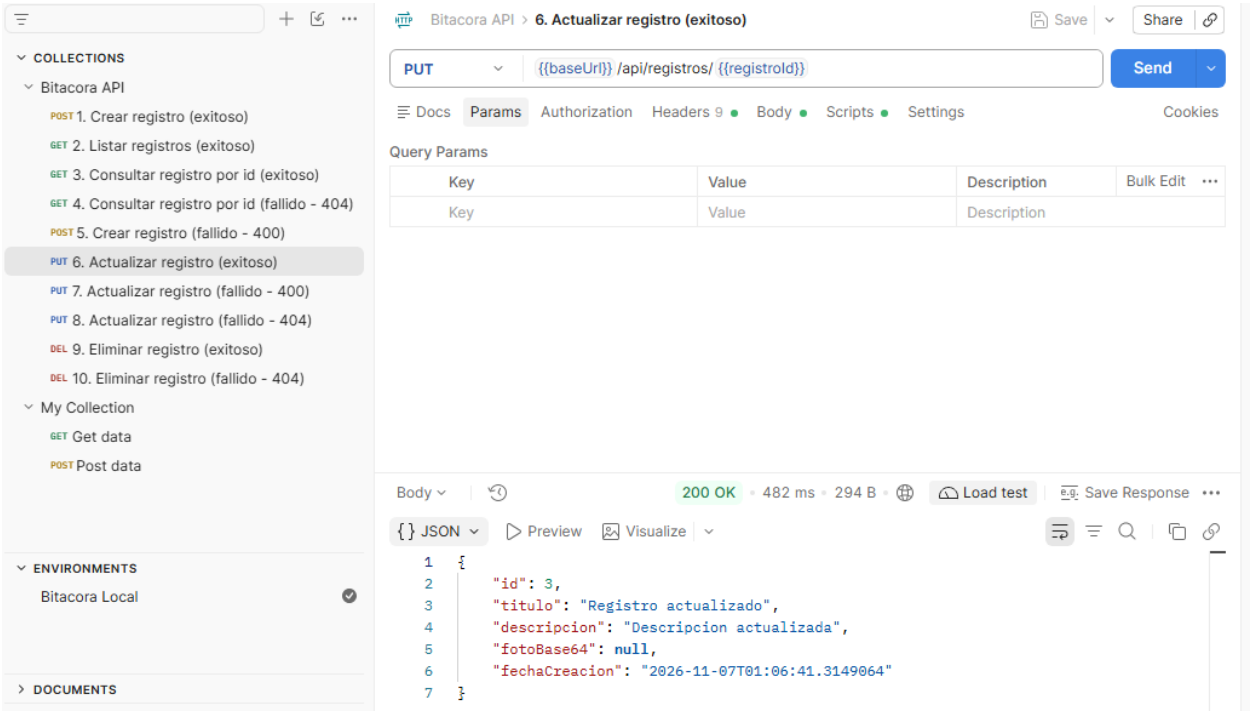
Consultar registro por id (fallido - 404)



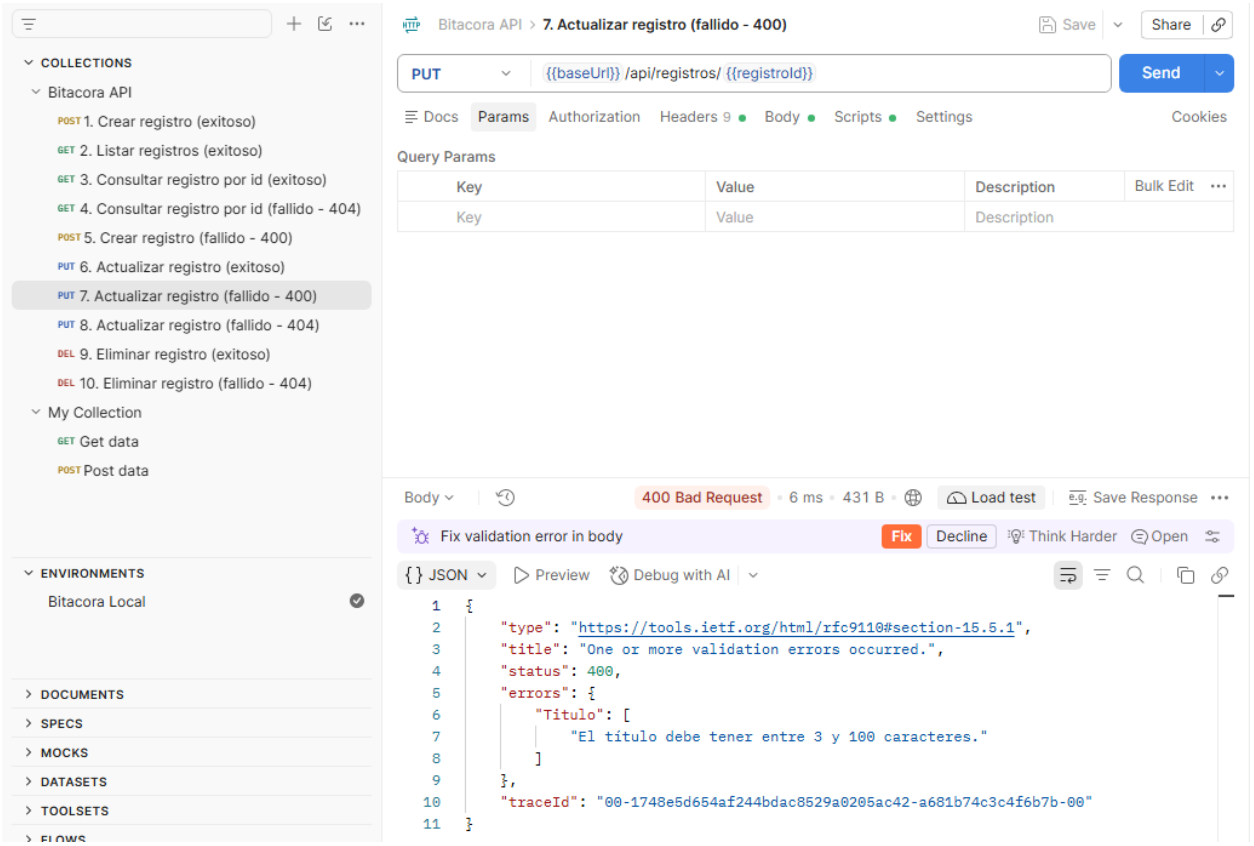
Crear registro (fallido - 400)



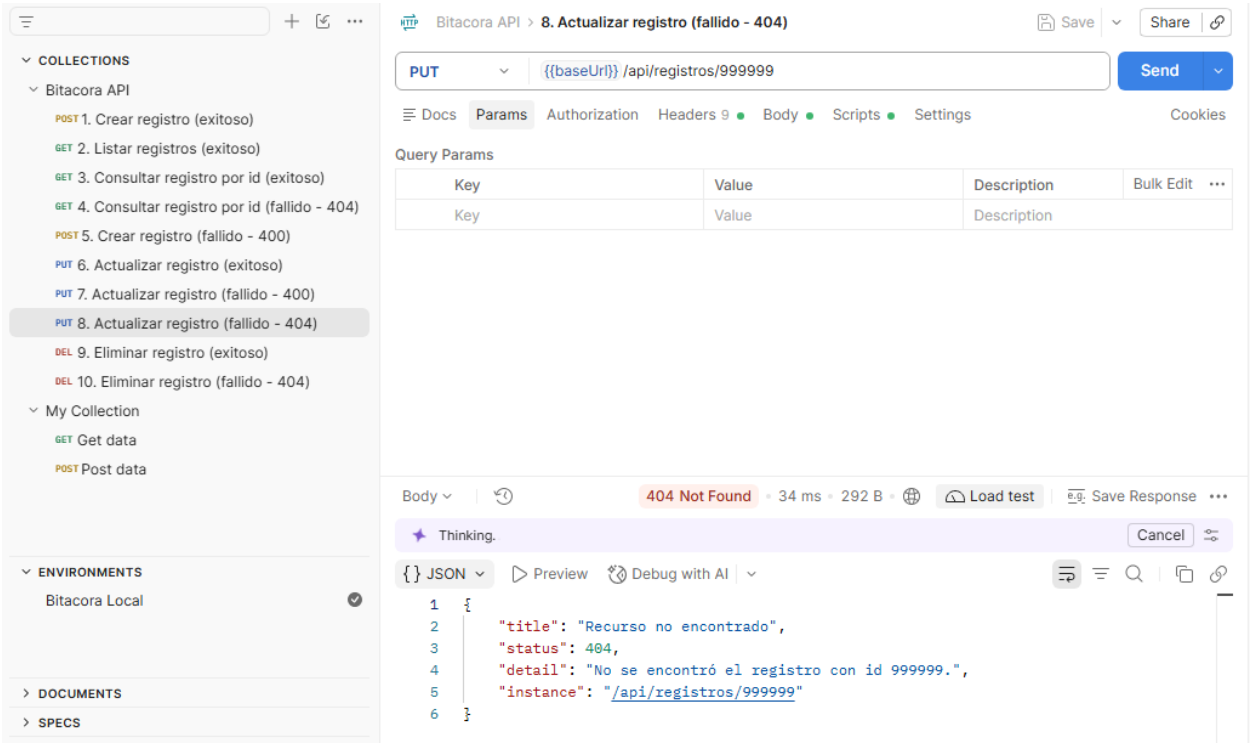
Actualizar registro (exitoso)



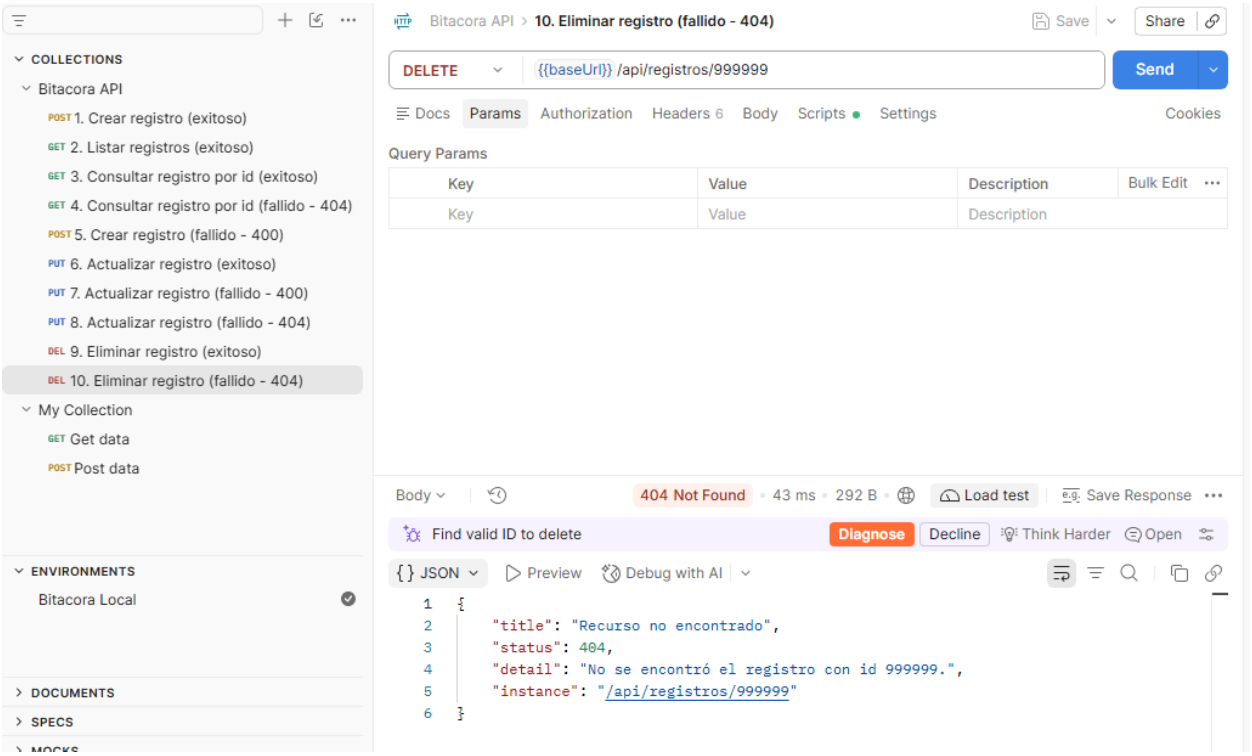
Actualizar registro (fallido - 400)



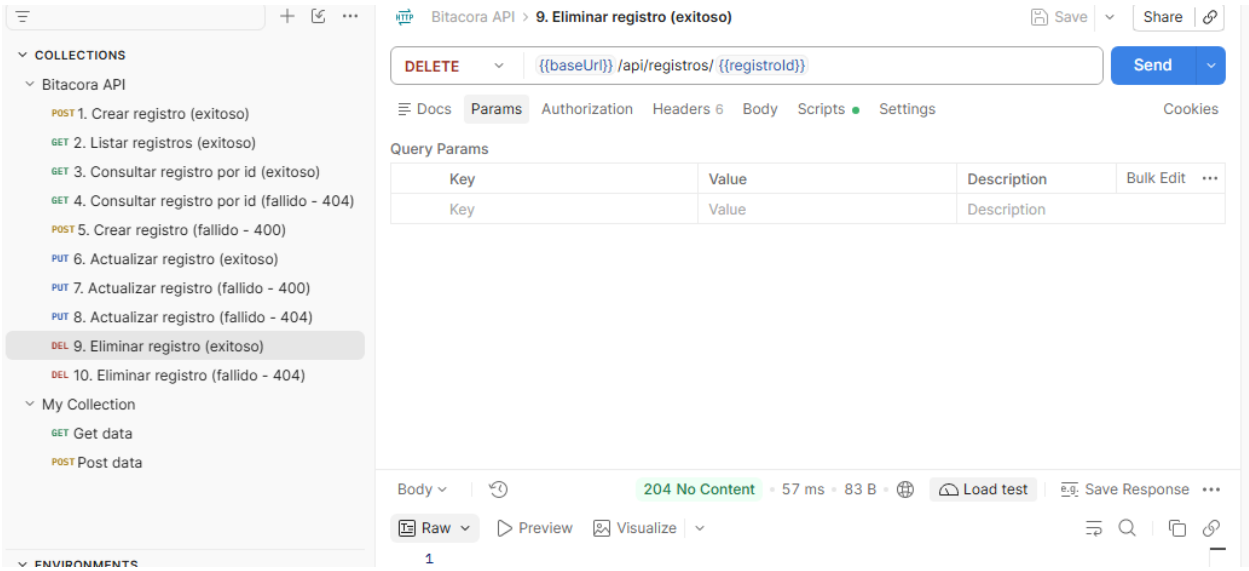
Actualizar registro (fallido - 404)



Eliminar registro (fallido - 404)



Eliminar registro (exitoso)



5. Distribución del trabajo entre los integrantes

Integrante	Número de cuenta	Responsabilidad
Angel Manuel Rivera	202330010471	Estructura de la solución, proyectos y referencias
Anthony Joel Gonzales Juárez	202310010114	Entidad de dominio y DTOs con validaciones
Brayan Wilfredo Torres Aguilar	201820030041	Repositorio EF Core, DbContext y migraciones
Johan Moises Lopez Moran	202220010502	Controladores, middleware de errores y configuración Swagger
Luis David Díaz Medina	202210010189	Colección de Postman, README y documentación

6. Conclusion:

Logramos construir y probar nuestra API siguiendo la estructura por capas, lo que facilita su mantenimiento. Quedaron funcionando todas las operaciones, con respuestas claras y seguras. La documentación y las pruebas confirman que todo funciona bien. Este trabajo es la base para las próximas etapas, donde conectaremos la aplicación Android y agregaremos seguridad al sistema.